

Q1

[org 0x0100]

jmp start

msg_a: db 'Hi, You pressed a', 0

msg_b: db 'Hi, You pressed b', 0

msg_wrong: db 'Hi, You entered wrong credentials', 0

start:

mov ah, 0x00

int 0x16

call ClearScreen

mov ah, 0x00

int 0x16

cmp al, 'a'

je pressed_a

cmp al, 'b'

je pressed_b

jmp wrong_key

pressed_a:

mov si, msg_a

call PrintString

jmp exit_program

pressed_b:

mov si, msg_b

call PrintString

jmp exit_program

wrong_key:

mov si, msg_wrong

call PrintString

exit_program:

mov ax, 0x4c00

int 0x21

ClearScreen:

push bp

mov bp, sp

push ax

push bx

push cx

```

push dx
mov ah, 0x06 ; Scroll up function
mov al, 0x00 ; Clear entire screen
mov bh, 0x07 ; Normal attribute (white on black)
mov ch, 0 ; Top row
mov cl, 0 ; Left column
mov dh, 24 ; Bottom row
mov dl, 79 ; Right column
int 0x10
; Move cursor to top-left corner
mov ah, 0x02 ; Set cursor position
mov bh, 0x00 ; Page number
mov dh, 0 ; Row
mov dl, 0 ; Column
int 0x10
pop dx
pop cx
pop bx
pop ax
pop bp
ret

```

```

PrintString:
push bp
mov bp, sp
push ax
push bx
push si
ps_loop:
lodsb ; Load byte from [SI] into AL, increment SI
cmp al, 0 ; Check for null terminator
je ps_done
; Print character using BIOS interrupt
mov ah, 0x0E ; Teletype output
mov bh, 0x00 ; Page number
mov bl, 0x07 ; Text attribute (white on black)
int 0x10
jmp ps_loop
ps_done:
pop si
pop bx
pop ax
pop bp

```

Ret

Q2

```
[org 0x0100]
jmp start
oldkb: dd 0
string: db 'He has food and drinks', 0
buffer: times 100 db 0

kbisr:
push ax
push bx
push cx
push dx
push si
push di
push es
push ds
push cs
pop ds ; Set DS to CS for data access
in al, 0x60 ; Read scan code from keyboard port
cmp al, 0x2A
je left_shift_pressed
cmp al, 0x36
je right_shift_pressed
cmp al, 0xAA
je shift_released
; Check for Right Shift release (scan code 0xB6 = 0x36 + 0x80)
cmp al, 0xB6
je shift_released
; For any other key, pass to original ISR
jmp pass_to_original
left_shift_pressed:
; Replace "and" with "dna" and display
call ClearScreen
call CopyAndReplace_And
mov si, buffer

call PrintString
jmp acknowledge_interrupt
right_shift_pressed:
; Replace "has" with "sah" and display
call ClearScreen
call CopyAndReplace_Has
mov si, buffer
```

```

call PrintString
jmp acknowledge_interrupt
shift_released:
; Clear screen when shift is released
call ClearScreen
jmp acknowledge_interrupt
acknowledge_interrupt:
; Send EOI (End of Interrupt) to PIC
mov al, 0x20
out 0x20, al
pop ds
pop es
pop di
pop si
pop dx
pop cx
pop bx
pop ax
iret
pass_to_original:
pop ds
pop es
pop di
pop si
pop dx
pop cx
pop bx
pop ax
; Jump to original ISR
jmp far [cs:oldkb]
; Subroutine: Copy string and replace "and" with "dna"
CopyAndReplace_And:
push ax
push si
push di
push cx
mov si, string
mov di, buffer
car_loop:
mov al, [si] ; Get current character
cmp al, 0 ; Check for end of string

je car_done
; Check if we have "and"

```

```

cmp al, 'a'
jne car_copy
; Check if next chars are "nd"
cmp byte [si+1], 'n'
jne car_copy
cmp byte [si+2], 'd'
jne car_copy
; Check word boundary (what comes after "and")
mov cl, [si+3]
cmp cl, 0
je car_replace_and
cmp cl, ' '
je car_replace_and
cmp cl, ','
je car_replace_and
jmp car_copy
car_replace_and:
; Write "dna" instead of "and"
mov byte [di], 'd'
inc di
mov byte [di], 'n'
inc di
mov byte [di], 'a'
inc di
add si, 3 ; Skip "and" in source
jmp car_loop
car_copy:
mov [di], al ; Copy character
inc di
inc si
jmp car_loop
car_done:
mov byte [di], 0 ; Null terminate
pop cx
pop di
pop si
pop ax
ret
; Subroutine: Copy string and replace "has" with "sah"
CopyAndReplace_Has:
push ax
push si
push di
push cx

```

```
mov si, string
mov di, buffer
```

```
crh_loop:
mov al, [si] ; Get current character
cmp al, 0 ; Check for end of string
je crh_done
; Check if we have "has"
cmp al, 'h'
jne crh_copy
; Check if next chars are "as"
cmp byte [si+1], 'a'
jne crh_copy
cmp byte [si+2], 's'
jne crh_copy
; Check word boundary (what comes after "has")
mov cl, [si+3]
cmp cl, 0
je crh_replace_has
cmp cl, ' '
je crh_replace_has
cmp cl, ','
je crh_replace_has
jmp crh_copy
crh_replace_has:
; Write "sah" instead of "has"
mov byte [di], 's'
inc di
mov byte [di], 'a'
inc di
mov byte [di], 'h'
inc di
add si, 3 ; Skip "has" in source
jmp crh_loop
crh_copy:
mov [di], al ; Copy character
inc di
inc si
jmp crh_loop
crh_done:
mov byte [di], 0 ; Null terminate
pop cx
pop di
pop si
```

```

pop ax
ret
; Subroutine: Clear screen
ClearScreen:
push ax
push cx
push di
push es

push 0xb800
pop es
xor di, di
mov ax, 0x0720 ; Space with normal attribute
mov cx, 2000 ; 80*25
cld
rep stosw
pop es
pop di
pop cx
pop ax
ret
; Subroutine: Print string at row 12, col 25
; SI = pointer to string
PrintString:
push ax
push bx
push di
push es
push 0xb800
pop es
; Calculate position: row 12, column 25
; Position = (row * 80 + col) * 2
mov ax, 12 ; row
mov bx, 80
mul bx ; ax = 12 * 80
add ax, 25 ; add column
shl ax, 1 ; multiply by 2 for byte offset
mov di, ax
mov ah, 0x0E ; Yellow on black (visible)
cld
ps_loop:
lodsb ; Load character from [SI]
cmp al, 0 ; Check for null terminator
je ps_done

```



```

mov [es:di], al ; Store character
mov [es:di+1], ah ; Store attribute
add di, 2 ; Move to next character position
jmp ps_loop
ps_done:
pop es
pop di
pop bx
pop ax
ret
start:
; Save old keyboard interrupt vector (INT 9)
xor ax, ax

mov es, ax
mov ax, [es:9*4]
mov [oldkb], ax
mov ax, [es:9*4+2]
mov [oldkb+2], ax
; Disable interrupts while hooking
cli
; Install our ISR
mov word [es:9*4], kbisr
mov [es:9*4+2], cs
; Enable interrupts
sti
; Print installation message
mov dx, msg
mov ah, 0x09
int 0x21
; Terminate and Stay Resident
; Keep program resident: (end - start + 256 bytes PSP) / 16
mov dx, (end - start + 256 + 15) / 16
mov ax, 0x3100 ; TSR function
int 0x21
msg: db 'String: "He has food and drinks"', 13, 10
db 'Left Shift = Replace "and" with "dna"', 13, 10
db 'Right Shift = Replace "has" with "sah"', 13, 10
db 'Release = Clear screen', 13, 10, '$'
end:

```